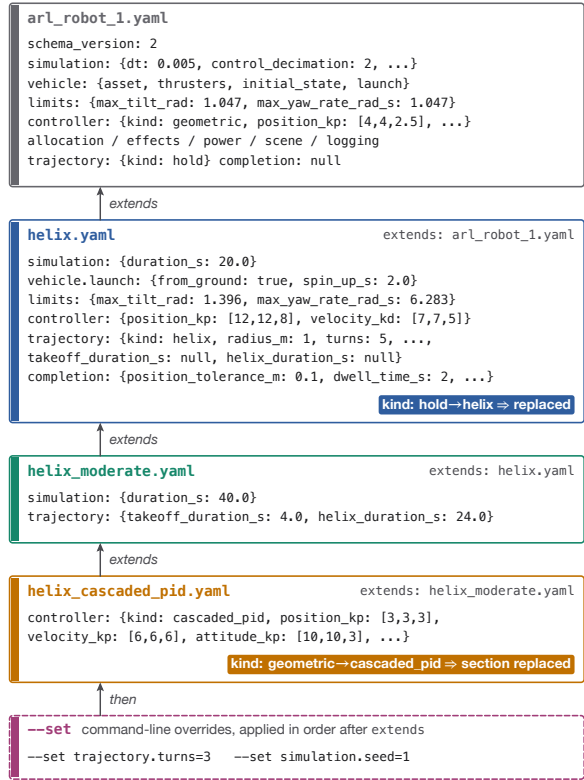


a One experiment = a YAML file that extends a parent + command-line overrides configs/, config/loader.py, config/schema.py



Merge rules config/loader.py

mapping + mapping merged key by key, recursively

list or scalar child value replaces parent value

different kind: whole section replaced, no stale gains

--set a.b=v v parsed as YAML; parent path must exist

cycle in extends rejected (Circular extends)

value set by: ■ arl_robot_1 ■ helix ■ helix_moderate ■ helix_cascaded_pid ■ --set

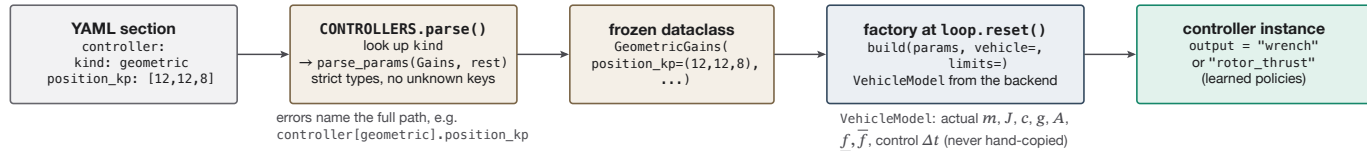
Resolved configuration (helix_cascaded_pid.yaml + two --set)

```
■ schema_version: 2
■ simulation.dt: 0.005 control_decimation: 2
■ simulation.duration_s: 40.0
■ simulation.seed: 1
■ vehicle.launch: {from_ground: true, spin_up_s: 2.0}
■ vehicle.thrusters / asset / initial_state.pos
■ limits: {max_tilt_rad: 1.396, max_yaw_rate: 6.283}
■ controller.kind: cascaded_pid (+ its gains only)
■ trajectory.kind: helix radius_m: 1 climb: 5
■ trajectory.turns: 3
■ trajectory.takeoff/helix_duration_s: 4.0 / 24.0
■ completion: {position_tolerance_m: 0.1, ...}
■ allocation, effects, power, scene, logging
```

validate_config(): unknown / duplicate keys, NaN, wrong shapes, booleans as numbers, and cross-field rules are all rejected → config.json

The resolved dict is what every module receives. It is written to runs/<id>/config.json; metadata.json adds the git commit, dirty flag and argv. python -m isaac_drone validate checks it without Isaac Sim.

b A plugin is a parameter dataclass plus a registered factory core/registry.py, core/params.py



CONTROLLERS Registry

kind	params dataclass	what it is
geometric	GeometricGains	Lee SO(3) + jerk/snap feedforward
cascaded_pid	CascadedPidGains	pos → vel → att → rate

TRAJECTORIES Registry

kind	params dataclass	what it is
hold	HoldParams	fixed CoM target and heading
helix	HelixParams	takeoff + helix, min-time optional

Extension points (Python Protocols / base classes; implement, register or inject)

interface	defined in	methods / contract	how it is plugged in
Controller	control/base.py	reset · compute(state, setpoint, dt) · notify_allocation · diagnostics	registry or MotionControlLoop(controller=)
SegmentedTrajectory	trajectories/base.py	_build(p_0, ψ_0) → [Segment(setpoint(σ), length, $\Delta\psi$, T None)]	registry; gets min-time plan, screening, phases
EffectModel	effects/pipeline.py	reset(seed) · evaluate(state, dt, wind) → Wrench about the CoM	EffectsPipeline → MotionControlLoop(effects=)
StateProvider	sim/base.py	reset · read_state(t) → VehicleState (sensor / estimator)	MotionControlLoop(state_provider=)
PowerSystem	power/system.py	LoadModel.current_a · ActuatorEnvelope.thrust_bounds_n (calibrated)	MotionControlLoop(power=)
RunHook	runtime/runner.py	start · after_step · paused · summary · close · finalize	run_experiment(hooks=)
VehicleBackend	sim/base.py	reset · read_state · motor_parameters · apply_motor_speeds · telemetry · ...	new backend + SimulationDriver